



IIE Transactions

Publication details, including instructions for authors and subscription information:

<http://www.tandfonline.com/loi/uiie20>

An order batching algorithm for wave picking in a parallel-aisle warehouse

A.J.R.M. (NOUD) GADEMANN^{a b}, JEROEN P. VAN DEN BERG^c & HASSAN H. VAN DER HOFF^d

^a Center for Production, Logistics and Operations Management, University of Twente, PO Box 217, Enschede, AE, 7500, The Netherlands E-mail:

^b ORTEC Consultants bv, PO Box 490, Gouda, AL, 2800, The Netherlands

^c Berenschot Management Consultants, Distribution Logistics Group, PO Box 8039, Utrecht, RA, 3503, The Netherlands E-mail:

^d Holland Raikonsult, PO Box 2855, Utrecht, GW, 3500, The Netherlands E-mail:

Published online: 27 Apr 2007.

To cite this article: A.J.R.M. (NOUD) GADEMANN, JEROEN P. VAN DEN BERG & HASSAN H. VAN DER HOFF (2001) An order batching algorithm for wave picking in a parallel-aisle warehouse, IIE Transactions, 33:5, 385-398, DOI: [10.1080/07408170108936837](https://doi.org/10.1080/07408170108936837)

To link to this article: <http://dx.doi.org/10.1080/07408170108936837>

PLEASE SCROLL DOWN FOR ARTICLE

Taylor & Francis makes every effort to ensure the accuracy of all the information (the "Content") contained in the publications on our platform. However, Taylor & Francis, our agents, and our licensors make no representations or warranties whatsoever as to the accuracy, completeness, or suitability for any purpose of the Content. Any opinions and views expressed in this publication are the opinions and views of the authors, and are not the views of or endorsed by Taylor & Francis. The accuracy of the Content should not be relied upon and should be independently verified with primary sources of information. Taylor and Francis shall not be liable for any losses, actions, claims, proceedings, demands, costs, expenses, damages, and other liabilities whatsoever or howsoever caused arising directly or indirectly in connection with, in relation to or arising out of the use of the Content.

This article may be used for research, teaching, and private study purposes. Any substantial or systematic reproduction, redistribution, reselling, loan, sub-licensing, systematic supply, or distribution in any form to anyone is expressly forbidden. Terms & Conditions of access and use can be found at <http://www.tandfonline.com/page/terms-and-conditions>

An order batching algorithm for wave picking in a parallel-aisle warehouse

A.J.R.M. (NOUD) GADEMANN¹, JEROEN P. VAN DEN BERG² and HASSAN H. VAN DER HOFF³

¹Center for Production, Logistics and Operations Management, University of Twente, PO Box 217, 7500 AE Enschede, The Netherlands and ORTEC Consultants bv, PO Box 490, 2800 AL Gouda, The Netherlands

E-mail: ngademann@ortec.nl

²Berenschot Management Consultants, Distribution Logistics Group, PO Box 8039, 3503 RA Utrecht, The Netherlands

E-mail: jpb@berenschot.com

³Holland Railconsult, PO Box 2855, 3500 GW Utrecht, The Netherlands

E-mail: hassan@casema.net

Received May 1999 and accepted June 2000

In this paper we address the problem of batching orders in a parallel-aisle warehouse, with the objective to minimize the maximum lead time of any of the batches. This is a typical objective for a wave picking operation. Many heuristics have been suggested to solve order batching problems. We present a branch-and-bound algorithm to solve this problem exactly. An initial upper bound for the branch-and-bound algorithm is obtained using a 2-opt heuristic. We present a basic version of the algorithm and show that major improvements are obtained by a simple but very powerful preprocessing step and an improved lower bound. The improvements for the algorithm are developed and tested using a relatively small test set. Next, the improved algorithm is tested on an extensive test set. It appears that problems of moderate size can be solved to optimality in practical time, especially when the number of batches is of importance. The 2-opt heuristic appears to be very powerful, providing tight upper bounds. Therefore, a truncated branch-and-bound algorithm would suffice in practice.

1. Introduction

In many warehouses the number of items per pick is less than that of a stored (unit) load. When individual orders consist of relatively few items, efficient order picking can often only be achieved by grouping orders into *batches*. Then a batch of orders can be picked in one tour through the warehouse and the mean travel time per order will be reduced. The *order batching problem* is the problem of distributing the orders among batches to enable efficient order picking, and of determining a batch picking tour for every batch. There are several variants of this problem, depending on the environment where the orders have to be picked (e.g., manual, semi-automatic or automatic picking, the capacity of the pick vehicle) and the objective of the order picking process. Sometimes the *sort-while-pick strategy* can be applied, for example when all orders consist of relatively few small items. In that case an order picker can use a pick cart carrying a number of bins, such that items belonging to the same order can be disposed into the same bin. The advantage of this strategy is that no sorting is needed after the orders have been picked. But in some cases the *pick-and-sort* strategy needs to be applied, for example to use the storage capacity of a pick

cart efficiently. Then all orders in a batch are disposed into one bin and the individual orders need to be sorted afterwards.

There are basically two criteria for order batching: the proximity of pick locations and the time window (Choe, 1991). *Proximity batching* minimizes travel time and thereby throughput, whereas *time window batching* optimizes due date performance. Many heuristics have been presented in the literature for proximity batching. Most of these heuristics first select a *seed order* for a batch and subsequently expand the batch with orders that have ‘proximity’ to the seed order as long as the vehicle capacity is not exceeded. The distinctive factor is the measure for the proximity of orders.

Armstrong *et al.* (1979) consider proximity batching with fixed batch sizes and present an Integer Programming model. Elsayed (1981), Elsayed and Stern (1983), Gibson and Sharp (1992) and Rosenwein (1996) consider proximity batching in a parallel-aisle warehouse. Hwang *et al.* (1988), Hwang and Lee (1988), Elsayed and Unal (1989) and Gibson and Sharp (1992) consider order batching in an order picking operation with man aboard Storage and Retrieval (S/R) machines. Pan and Liu (1995) compare the batching heuristics presented in the

literature for man aboard S/R machine operations and recommend the economic convex hull heuristic of Hwang and Lee (1988). De Koster *et al.* (1999) compare several heuristics for the order batching problem in a parallel-aisle warehouse. For the subproblem of determining an optimal tour for a given batch, which can efficiently be solved by the dynamic programming algorithm of Ratliff and Rosenthal (1983), they use the largest gap heuristic and the S-shape heuristic.

Elsayed *et al.* (1993) consider time window batching. They present a heuristic for the batching of orders with due dates. The objective is to minimize earliness and tardiness penalties. Elsayed and Lee (1996) consider sequencing and batching of storage and retrieval requests to tours such that the total tardiness of the retrieval requests is minimized.

In this paper we consider batching in a *wave picking operation*. Wave picking means that a number of batches, a *wave*, is picked simultaneously by a group of order pickers. Every order picker picks one batch and all order pickers start at the same time. We assume that the sort-while-pick strategy is applied in a parallel-aisle warehouse so that orders do not need to be accumulated and sorted after picking. We present an exact algorithm that assigns orders to batches such that the maximum lead time for any of the batches is minimized, which is a typical objective for a wave picking operation.

2. Problem description and assumptions

We consider the order batching problem for a warehouse with parallel aisles which are connected by cross aisles at the front and the back of each aisle. An example of such a warehouse is given in Fig. 1.

An *order* consists of one or multiple *order lines*. Each order line specifies the requested number of items of a certain product. Products are stored on storage modules such as pallets or cartons. Typically much less than a

stored load of a product is requested, e.g., only one or a few items. The orders are picked manually by order pickers using pick carts. Each of these pick carts carries an identical number of bins. A *batch* is the set of orders that is picked by one order picker. The *size* of a batch is the number of orders in the batch. We consider a wave picking operation where orders are picked according to the sort-while-pick strategy. Wave picking is applied when several order pickers simultaneously handle a larger set of orders and the response time for the complete set is important. This occurs for example when a set of orders together forms a truck load. If the expedition area and the number of loading docks are limited, it is advantageous to handle the trucks dock by dock. A truck cannot leave before all orders for that truck have been picked. Therefore, the lead time of a wave determines the (earliest) departure time of a truck. We require that every order is picked completely by one order picker and that every bin contains items for a single order only. The *lead time* of a batch (or an order) is the sum of the *travel time* to travel along the pick locations plus the *extraction time*, which includes time for extracting an item from the pick location, as well as time for counting, searching, acceleration and deceleration of the pick cart, administration, walking towards the rack, etc.

Let n be the number of orders, m the number of batches, and b the number of bins on a pick cart, i.e., the maximum batch size. Order $i \in \{1, 2, \dots, n\}$ will often simply be denoted by i . B_j denotes batch j , i.e., the set of orders in the j th batch, $j \in \{1, 2, \dots, m\}$. $B_j \cup \{i\}$ is the batch that is obtained by adding order i to batch B_j . e_i denotes the total extraction time for order i . $|X|$ denotes the cardinality of a set X . Let S be the set of all possible batches, and let a_s be the incidence (column) vector for batch $s \in S$, i.e., $a_{is} = 1$ if and only if batch s contains order i . Note that every a_s has at most b non-zero entries. Let l_s denote the lead time for batch s . The order batching problem can now be formulated as follows:

$$\min \max_{s \in S} l_s x_s,$$

such that

$$\sum_{s \in S} a_{is} x_s = 1, \quad i = 1, 2, \dots, n,$$

$$x_s \in \{0, 1\},$$

or, in other words, partition a given set of n orders into m batches of maximum size b , such that the maximum lead time is minimized. We make the following assumptions.

1. An order picking tour starts and ends at the I/O station.
2. Every order fits into one bin.
3. Order pickers do not cause congestion.
4. The extraction time is given for every pick.

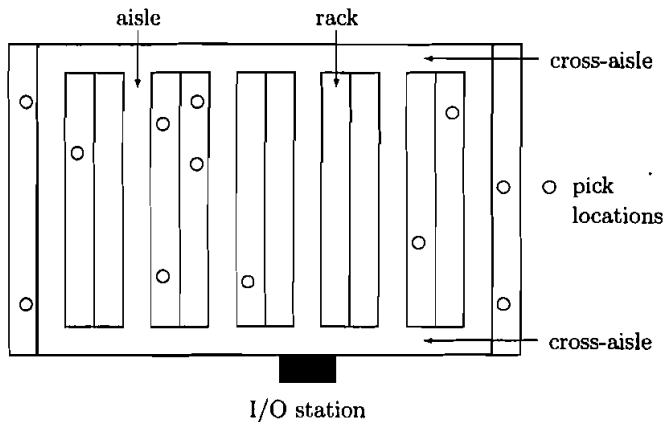


Fig. 1. Configuration of a parallel-aisle warehouse.

5. The velocity of an order picker is constant and given.
6. All batches have size b .

Assumption 6 is not restrictive, since, if the number of orders in a wave is not a multiple of b , we can always add dummy orders without picks to the wave. However, in most practical instances sufficient orders are known, and a multiple of b orders will be picked in a wave to use the capacity of the order pickers and the pick carts efficiently. Assumption 5 implies that the travel time is linear in the length of a tour. Observe that, due to Assumptions 1, 4 and 5, for a given batch an optimal tour is given by a tour through the warehouse of minimum length, starting and ending at the I/O station and visiting all the pick locations in the batch. We have assumed that the order pickers do not cause congestion. The impact of congestion is highly dependent on the number of order pickers, the warehouse layout, and the pick density. However, in many warehouses order pick carts can pass each other within an aisle. Therefore, the impact of congestion may be rather small. Within our framework, congestion is very hard to model. If congestion is expected to have a significant impact, the operational control should for example be tested by simulation. In any case, our model will give a good indication of how to batch the orders.

Ratliff and Rosenthal (1983) show that the problem of determining an optimal order picking tour along a given set of locations in a parallel-aisle warehouse, which is a special case of the traveling salesman problem, can be solved efficiently. They present a dynamic programming algorithm which runs in $\mathcal{O}(na + np)$ time if the picks in the problem instance are sorted, where na is the number of aisles and np the number of picks. Their algorithm is based on the observation that in an optimal tour there are only six configurations to travel within aisles and only four configurations to travel between aisles through the cross aisles. These configurations are schematically given in Figs. 2 and 3.

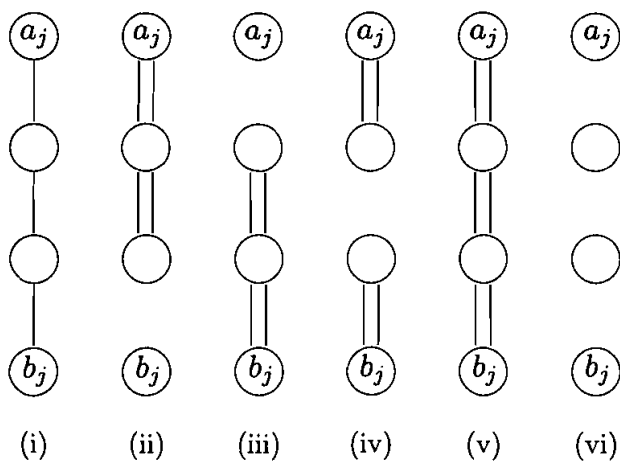


Fig. 2. Possible arc configurations for any aisle j in an optimal tour.

Nodes b_j and a_j represent the beginning and end points of aisle j . The other nodes represent pick locations in an aisle. The gap in configuration (iv) of Fig. 2 occurs between the two consecutive picks with the largest gap in between. Configuration (v) may occur if there are pick locations in the cross aisles. In that case an aisle may be traversed completely, the cross-aisle is entered to visit a pick location, and the same aisle is traversed again to return to the other cross-aisle. Configuration (vi) simply depicts the situation that an aisle without locations that must be visited is passed but not entered. An optimal tour is constructed by efficiently examining the possible configurations.

Determining an optimal order picking tour is polynomially solvable, but the order batching problem is $\mathcal{NP}$ -hard. We give a polynomial reduction from PARTITION (Garey and Johnson, 1979) to show that the decision variant of the order batching problem is $\mathcal{NP}$ -complete. The decision variant of the order batching problem with n orders and a given batch size b , is the question whether there exists a partition of the n orders into batches, with each batch containing b orders, such that the maximum lead time of the batches is smaller than or equal to a given constant y .

Theorem 2.1. *The decision variant of the order batching problem is $\mathcal{NP}$ -complete.*

Proof. First of all, it is clear that the problem is a member of the class $\mathcal{NP}$ – for any partitioning of the n jobs into batches, we can check in polynomial time using Ratliff and Rosenthal's algorithm whether the maximum lead time of all batches is smaller than or equal to the given constant y .

We now prove that PARTITION is polynomially reducible to our problem.

PARTITION: Given a finite set A and a size $s(a) \in \mathbb{Z}^+$ for each $a \in A$, is there a subset $A' \subseteq A$ such that

$$y = \sum_{a \in A'} s(a) = \sum_{a \in A - A'} s(a)?$$

PARTITION is known to be $\mathcal{NP}$ -complete (Garey and Johnson, 1979). We transform this problem into an instance of the order batching problem as follows. For every $a \in A$, we define an order with $s(a)$ picks. Let this be $k = |A|$ orders. Furthermore we define k dummy orders that contain no picks. Let the extraction time per pick be one and the travel time be zero. It is clear that PARTITION has a solution if and only if the order batching problem with two batches of size k has a solution with value smaller than or equal to y . ■

3. A basic branch-and-bound algorithm

In this section we present a basic branch-and-bound algorithm for solving the order batching problem. We

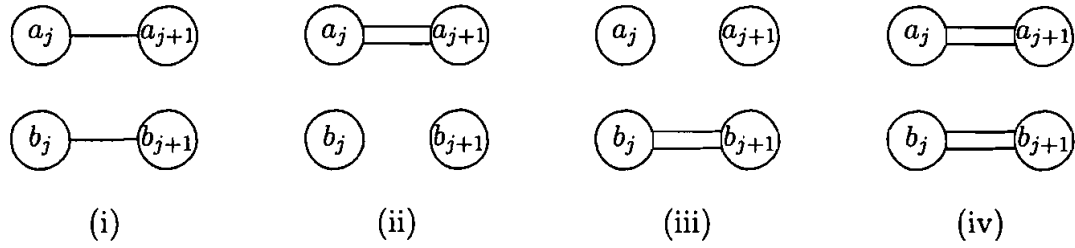


Fig. 3. Possible arc configurations for cross-over between aisle j and aisle $j+1$ in an optimal tour.

discuss the branching strategy, an initial upper bound and a straightforward lower bound. In the next section improvements on the basic version will be discussed.

First we describe the branch-and-bound *search tree*. The search tree consists of n levels. At level i order i is assigned to a batch as follows. Let the batches in a node be denoted by $B_1, B_2, \dots, B_m$. In the root (level 1) we assign order 1 to batch B_1 . All other batches are empty. Let d denote the number of non-empty batches in a node at level $i-1$. We call a batch *complete* if it has size b . We add a branch for every non-empty batch that is not complete, and in every branch order i is assigned to the corresponding batch. If $d < m$ we also add a branch in which we assign order i to the first empty batch B_{d+1} . The structure of the branch-and-bound tree for a problem with nine orders and batches with a size of three is given in Fig. 4.

By construction, every node of the search tree corresponds to a unique subproblem. This can be seen as follows. Assume that we have d non-empty batches $B_1, B_2, \dots, B_d$ in a node at level i . Let $B_{j,[k]}$ denote the k th order of batch B_j and let $\phi(B_{j,[k]})$ be its order number. By construction we have

$$\phi(B_{j,[1]}) < \phi(B_{j,[2]}) < \dots < \phi(B_{j,[|B_j|]}), \quad \forall j,$$

and

$$\phi(B_{1,[1]}) < \phi(B_{2,[1]}) < \dots < \phi(B_{d,[1]}).$$

Since any exchange of two orders in a batch or two batches in a node disturbs these orderings, only one permutation occurs in the search tree.

The optimal solution to the order batching problem can be found by evaluating all nodes at level n . By computing an upper bound UB on the optimal solution of the problem and a lower bound LB in a node of the search tree, a node can be fathomed if $LB \geq UB$, and less nodes need to be evaluated.

We apply the *depth-first* branching strategy. We compute an initial upper bound by a *2-opt* heuristic. This local search heuristic proceeds as follows. For a given partition of the orders into batches, we call a batch that constitutes the largest lead time a *bottleneck batch*. Obviously a partition cannot be improved without altering a bottleneck batch. Therefore we define the *neighborhood* of a partition as the set of all partitions that can be obtained

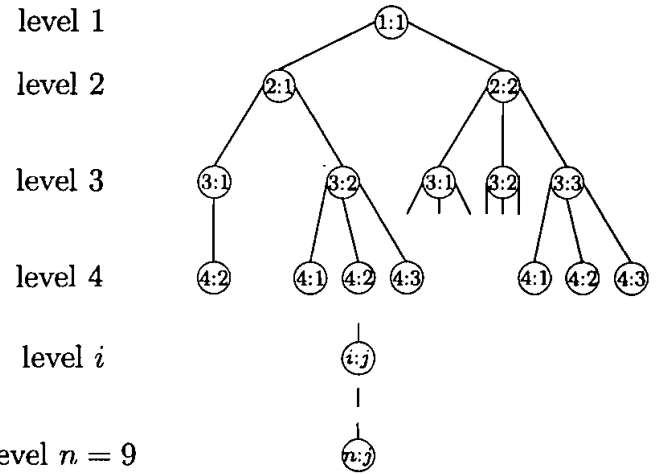


Fig. 4. The structure of the branch-and-bound tree for $n = 9$ and $b = 3$. $i:j$ means that order i is added to batch j .

by exchanging an order from a bottleneck batch with an order from any of the other batches. In the *2-opt* heuristic we evaluate neighbors of the current partition and accept the first improvement that we encounter. To determine the optimal tour for a batch we use the algorithm of Ratliff and Rosenthal (1983). We start the heuristic with a random partition of the orders into batches. The heuristic stops when none of the neighbors of the current solution gives an improvement. Since this heuristic runs very fast, we may rerun it several times with different initial partitions and take the best solution as initial upper bound. The results for the *2-opt* heuristic will be discussed in Section 6.

A straightforward lower bound for a node is obtained by taking the maximum of the lead times for every batch in that node. Note that batches at level $i < n$ may contain less than b orders. The lower bound can be computed by determining the lead time for every batch using the algorithm of Ratliff and Rosenthal (1983).

4. Improvements

In the previous section we have described a basic branch-and-bound algorithm. In this section we discuss several improvements of the basic algorithm in detail.

4.1. Preprocessing: rearranging the input data

In Section 6 the initial upper bounds provided by 2-opt will prove to be tight. Therefore, improving the upper bound is not our first concern. Despite this tight upper bound a node can only be fathomed if it contains a batch with a large lower bound on its lead time. The lower bound will be discussed in more detail in Section 4.2. Irrespective of the implemented lower bound, we may expect that a node is more likely fathomed at a higher level i , in particular when one or more batches are (almost) complete. It may prove valuable to construct batches with large lead times at low levels of the search tree, so that the associated nodes can be fathomed and less nodes need to be evaluated. Since the orders are assigned to batches in order of increasing order number i , we may influence the construction of the batches by rearranging the set of orders. The following rearrangement strategies have been implemented:

- RAND** The orders are rearranged randomly.
- LOPT** The orders are rearranged in sequence of decreasing lead times.
- LBPT** The orders are rearranged based on decreasing lead times for batches of size two, as follows. Order 1 is the order with the largest lead time. Let $l_{s,t}$ be the lead time for a batch of orders s and t , and suppose that orders $1, 2, \dots, i$ have already been rearranged. Then the order $t \notin \{1, 2, \dots, i\}$ which maximizes

$$\sum_{s=1}^i l_{s,t},$$

becomes order $i+1$ in the rearrangement. So the next order t in the rearrangement is the order that on average constitutes the largest lead time for batches of size two if it is combined with the orders $1, 2, \dots, i$.

In all cases ties are broken arbitrarily. Note that every rearrangement requires at most $\mathcal{O}(n^2)$ evaluations of the lead time of a batch. With LOPT orders with large lead times are assigned to batches at low levels i of the search tree. Orders with larger lead times are more likely to constitute large batch lead times. With LBPT not only the order lead times are taken into account, but also orders that do not fit very well together are more likely to be combined at low levels of the search tree. The impact of the rearrangement strategies will be discussed in Section 6.

4.2. Lower bounds

By $LB(B_j)$ we denote a lower bound for batch B_j . If a batch is omitted, the lower bound for a node is meant, which is the maximum over all $LB(B_j)$ for the batches B_j in that node.

4.2.1. Lower bound 1 – based only on the lead time per batch for assigned orders

This is the lower bound as described in Section 3, where the travel time for every batch in a node is computed exactly using the algorithm of Ratliff and Rosenthal (1983). A minor variant of this lower bound can be obtained by computing a lower bound on the batch lead times. Since for a given order the extraction time is fixed, only the shortest order picking tour needs to be computed. Such a tour will visit the left most and the right most aisle that contains at least one pick location, and will possibly visit the aisles in between. A lower bound on the length of this optimal tour is obtained by taking for each aisle the cheapest of all aisle configurations of Fig. 2, increased by twice the travel time between every two consecutive aisles that are visited. The exact lower bound will be denoted by LB_{1e} , and the lower bound based on the aisle configurations will be denoted by LB_{1c} . We may compute LB_{1e} if $LB_{1c} < UB$, to fathom more nodes. Experiments with over 50 000 batches B_j of various sizes have shown that for approximately 95% of the batches the exact travel time is within 10% of the travel time based on LB_{1c} , which we denote by $LBTT(B_j)$. Therefore, if we compute $LB_{1c}(B_j)$, we evaluate $LB_{1e}(B_j)$ only if:

$$LB_{1c}(B_j) < UB < LB_{1c}(B_j) + 0.1 \times LBTT(B_j). \quad (1)$$

LB_1 is based on LB_{1e} or on a combination of LB_{1c} and LB_{1e} .

4.2.2. Lower bound 2 – based on the completion of a single batch

In a node at level i , LB_1 is solely based on orders $1, 2, \dots, i$. Let $MP(r, i)$ denote the minimum extraction time for any set of $r \in \{1, 2, \dots, n-i\}$ of the remaining orders, i.e., any set of r orders with index larger than i . All the $MP(r, i)$ may be calculated initially and hence need only be calculated once. Consider a batch B_j of size $b-r$, $r \in \{1, 2, \dots, b\}$. Completing B_j requires at least $MP(r, i)$ additional extraction time. Thus we can tighten LB_1 , for a batch B_j at level i , as follows:

$$LB_2(B_j) = LB_1(B_j) + MP(r, i).$$

4.2.3. Lower bound 3 – based on the specific assignment at the current level

If a batch B_j at level $i-1$ requires r more orders, $r \in \{1, 2, \dots, b-1\}$, then, by construction, we know that order i will be added to B_j at level i . This yields:

$$LB_3(B_j \cup \{i\}) = LB_1(B_j) + e_i + MP(r-1, i),$$

where e_i denotes the extraction time for order i .

4.2.4. Lower bound 4 – based on the completion of all batches

In a node at level i we not only know the number of orders required to complete a single batch, but also the

number $n - i$ of orders to complete all batches. Let P be the set of batches in the current node that are not complete. We sort the batches B_j in P in sequence of decreasing $LB_1(B_j)$ (ties are broken arbitrarily). Let $B_{[k]}$ denote the k th batch in that sequence and let $r(k)$ be the total number of orders required to complete batches $B_{[1]}, B_{[2]}, \dots, B_{[k]}$. Since we have assigned order i to batch B_j we find a lower bound LB_4 for the minimum wave lead time by:

$$LB_4 = \max_{k=1,2,\dots,|P|} \left\{ \frac{MP(r(k), i)}{k} + \frac{1}{k} \sum_{j=1}^k LB_1(B_{[j]}) + \delta_{k,i} e_i \right\}, \quad (2)$$

with

$$\delta_{k,i} = \begin{cases} 1 & \text{if } i \in B_{[j]} \text{ for some } j \leq k, \\ 0 & \text{otherwise.} \end{cases}$$

The first term on the right-hand-side of (2) represents the average extraction time that is at least required to complete the first k batches, the second term gives a lower bound on the average lead time over the first k batches.

Several combinations of these four lower bounds can be used. Suppose that all lower bounds are used. At level i order i is assigned to a batch B_j of level $i - 1$. We compute $LB_3(B_j \cup \{i\})$ and if necessary LB_4 , using $LB_1(B_j)$ which has already been computed at level $i - 1$. If the node cannot be fathomed, then $LB_1(B_j \cup \{i\})$ and $LB_2(B_j \cup \{i\})$ are computed. So first LB_3 is computed, then LB_4 , then LB_1 and finally LB_2 . We illustrate the lower bounds by an example. Eight orders must be partitioned into two batches of four orders. In a node at level 4 we have $B_1 = \{1, 2\}$, $B_2 = \{3, 4\}$. Orders 1, 2, 3 and 4 consist of three picks each, orders 5, 6, 7 and 8 consist of one, two, five and two picks respectively. The extraction time is one time unit per pick. The current upper bound UB equals 67. The batches have the characteristics, listed in Table 1.

Assume that only LB_1 and LB_2 are used. At level 5 $B_1 \cup \{5\}$ is evaluated. $LB_1(B_1 \cup \{5\}) = 66 < UB$, and we also compute $LB_2(B_1 \cup \{5\}) = 66 + MP(1, 5) = 68 > UB$. So this node is fathomed. For $B_2 \cup \{5\}$ we find $LB_1(B_2 \cup \{5\}) = 63 < UB$ and $LB_2(B_2 \cup \{5\}) = 63 + MP(1, 5) = 65 < UB$. This node is not fathomed at level 5.

Now also suppose that LB_3 and LB_4 are used to evaluate $B_2 \cup \{5\}$. Before computing $LB_1(B_2 \cup \{5\})$ we first compute

$$\begin{aligned} LB_3(B_2 \cup \{5\}) &= LB_1(B_2) + e_5 + MP(1, 5) \\ &= 61 + 1 + 2 = 64, \end{aligned}$$

and, since this is smaller than UB , we also compute LB_4 , yielding (for $k = 2$):

$$\begin{aligned} LB_4 &= MP(3, 5)/2 + (LB_1(B_1) + LB_1(B_2) + e_5)/2 \\ &= (9 + 126)/2 = 67.5 > UB. \end{aligned}$$

Thus the node is fathomed.

5. The test sets

In this section we discuss the test instances that are used in Sections 6 and 7 to test the performance of the 2-opt heuristic and the branch-and-bound algorithm. The test instances can be characterized by the following parameters:

- n = the number of orders;
- m = the number of batches;
- ws = the warehouse size, i.e., the total number of storage locations;
- na = the number of aisles;
- nl = the number of item locations per aisle side;
- cd = the cross-distance between two consecutive aisles;
- IO = the position of the IO-station (aisle, location);
- l_{ex} = interval for uniform sampling of the extraction time per pick;
- pA = probability to visit an aisle;
- pL = average probability to visit a location in an aisle that is visited;
- Ep = the expected number of picks per order.

We define a set of parameter settings to test the impact of several parameters. The settings are given in Table 2. The test sets are indicated by a code with the following interpretation. The first symbol indicates whether the settings are more or less fictive (F) or that they are comparable with most commonly encountered warehouse types in practice (P) — the latter are derived from de Koster *et al.* (1999). The second symbol specifies the storage policy within an aisle, where we distinguish between the randomized storage policy (R) and the ABC-storage policy (A). The third and fourth symbol give the test set number. Possibly further symbols denote that the same test set is used with several batch sizes (denoted by a, b, c and d).

By these settings we focus on several aspects. The main focus is on the problem size. We particularly vary the number of orders and the number of batches. This is done in sets 1 to 9. Set 6 often serves as a base set to study the impact of other characteristics. We study the impact of the warehouse layout (sets 6, 10 and 11), the pick density (sets 12, 14 and 15), family grouping (sets 6, 12 and 13),

Table 1. Characteristics of the batches

Batch B_j	$LB_{1c}(B_j)$	$LB_{1e}(B_j)$	$LB_1(B_j)$
B_1	63	64	64
B_2	61	61	61
$B_1 \cup \{5\}$	65	66	66
$B_2 \cup \{5\}$	63	63	63

Table 2. Parameter settings for the test sets

Test set	<i>n</i>	<i>m</i>	<i>ws</i>	<i>na</i>	<i>nl</i>	<i>cd</i>	<i>IO</i>	<i>I_{ex}</i>	<i>pA</i>	<i>pL</i>	<i>Ep</i>
FR01ab	15	3, 5	400	10	20	3	(5, 0)	[1, 3]	1	0.025	10
FR02	16	4	400	10	20	3	(5, 0)	[1, 3]	1	0.025	10
FR03ab	18	3, 6	400	10	20	3	(5, 0)	[1, 3]	1	0.025	10
FR04ab	20	4, 5	400	10	20	3	(5, 0)	[1, 3]	1	0.025	10
FR05	21	7	400	10	20	3	(5, 0)	[1, 3]	1	0.025	10
FR06abcd	24	3, 4, 6, 8	400	10	20	3	(5, 0)	[1, 3]	1	0.025	10
FR07	25	5	400	10	20	3	(5, 0)	[1, 3]	1	0.025	10
FR08abc	30	3, 5, 6	400	10	20	3	(5, 0)	[1, 3]	1	0.025	10
FR09	32	4	400	10	20	3	(5, 0)	[1, 3]	1	0.025	10
FR10abc	24	3, 4, 6	400	5	40	3	(3, 0)	[1, 3]	1	0.025	10
FR11abc	24	3, 4, 6	400	20	10	3	(10, 0)	[1, 3]	1	0.025	10
FR12abc	24	3, 4, 6	400	10	20	3	(5, 0)	[1, 3]	0.2	0.125	10
FR13abc	24	3, 4, 6	400	10	20	3	(5, 0)	[1, 3]	0.1	0.25	10
FR14abc	24	3, 4, 6	400	10	20	3	(5, 0)	[1, 3]	0.2	0.25	20
FR15abc	24	3, 4, 6	400	10	20	3	(5, 0)	[1, 3]	0.2	0.05	4
FR16abc	24	3, 4, 6	400	10	20	3	(5, 0)	[0, 1]	1	0.025	10
FR17abc	24	3, 4, 6	400	10	20	3	(5, 0)	[7, 11]	1	0.025	10
FA18abc	24	3, 4, 6	400	10	20	3	(5, 0)	[1, 3]	1	0.025	10
FA19abc	24	3, 4, 6	400	10	20	3	(5, 0)	[1, 3]	0.2	0.125	10
FA20abc	24	3, 4, 6	400	10	20	3	(5, 0)	[1, 3]	0.2	0.25	20
FA21abc	24	3, 4, 6	400	10	20	3	(5, 0)	[1, 3]	0.2	0.05	4
PR22	15	3	240	4	30	4	(2, 0)	[8, 12]	1	0.017	4
PR23	20	4	240	4	30	4	(2, 0)	[8, 12]	1	0.017	4
PR24abc	24	3, 4, 6	240	4	30	4	(2, 0)	[8, 12]	1	0.017	4
PR25	15	3	400	10	20	5	(5, 0)	[1, 3]	1	0.015	6
PR26	20	4	400	10	20	5	(5, 0)	[1, 3]	1	0.015	6
PR27abc	24	3, 4, 6	400	10	20	5	(5, 0)	[1, 3]	1	0.015	6

the storage policy (sets 6, 12, 14, 15 (randomized storage) versus 18, 19, 20, 21 (ABC-storage), respectively), and the extraction time (sets 6, 16 and 17). Furthermore we look at some practical parameter settings (sets 22 to 27).

All distances are in time units. The distance from a cross-aisle to the first location in an aisle and the distance between two consecutive aisle locations is one. The distance between two consecutive aisles is given by *cd*, the cross distance. The probability of a location to be visited depends on *pA* and *pL*. *pA* is related to family grouping. We assume a very simple policy. If family grouping is applied, we assume a one-to-one correspondence between aisles and families. Every aisle has the same probability *pA* to be visited. In the case of family grouping, *pA* is smaller than one. If an aisle is visited, then *pL* denotes the average probability for a location in that aisle to be visited. For randomized storage, the probability *pv_i* to visit a location in a visited aisle equals *pL* for every location. For the ABC-storage policy, *pv_i* is proportional to $1/d(i)$, where $d(i)$ denotes the distance from location *i* to the nearest cross-aisle. More precisely, if we define

$$\bar{d} = \frac{\sum_{i=1}^{nl} 1/d(i)}{nl},$$

then, for the ABC-storage policy, *pv_i* is given by:

$$pv_i = \frac{pL}{\bar{d}} \frac{1}{d(i)}.$$

Hence, the locations closer to a cross-aisle have a higher probability to be visited, which reflects a situation that fast movers are stored closest to the cross aisles. Finally, if a location is visited, the associated extraction time is uniformly sampled from *I_{ex}*.

In the next sections the test results are discussed. All tests reported in this paper are performed on a Pentium II/233 MHz/64 Mb RAM.

6. Preliminary test results

In this section we take a closer look at the branch-and-bound algorithm. We test several versions of the algorithm, combining several rearrangement strategies and lower bounds. We use one instance per parameter setting of Table 2. Based on these preliminary test results, the apparently best version with respect to computational efficiency is selected for more extensive testing in Section 7.

First we discuss the 2-opt heuristic. We ran the 2-opt heuristic $3n$ times, and compared the best objective

function value found after $1, 0.5n, n, 2n$ and $3n$ runs respectively. After n runs all instances were within 2% of the best value found after $3n$ runs, and within 3% of the optimal solution. Based on these results we will use n runs of the 2-opt heuristic to obtain the initial *UB*.

Next we discuss the results for the branch-and-bound algorithm. All CPU times for the branch-and-bound algorithm in this section and following sections do not include the CPU time for 2-opt. For the testing we allowed a maximum of 1800 CPU seconds per instance. For runs where the optimal solution is found within 1800 seconds, the CPU time is presented. Otherwise the branch-and-bound algorithm is truncated and the best objective function value found after 1800 seconds is given.

First we look at the impact of the rearrangement of the input data as described in Section 1. In the comparison of these rearrangement strategies a combination of all lower bounds is used – more details of this implementation follow later in this section. Some typical test results are given in Table 3.

The second column gives the initial upper bound, obtained after n runs of 2-opt. The third column gives the optimal objective function value (or the best objective function value known). In the next three columns the computation times using the different rearrangement strategies are given for those instances that were solved within 1800 seconds. For the unsolved instances, indicated by an *, the best objective function value found is given. The last column gives the computation time for n runs of 2-opt. The last row gives the number of instances

that were solved within 1800 seconds, out of a total of 63 instances.

The table shows that the impact is huge. LOPT yields a reduction of CPU time compared to RAND of over 90% for smaller instances and up to 99% for larger instances. Also the number of instances that were solved increases from 20 to 50. LBPT yields another substantial CPU time reduction over LOPT for most instances, whilst in only six out of 63 instances did the CPU time slightly increase. Another two instances are solved. However, 11 instances remained unsolved within the 1800 seconds (6d, 8c, 10c, 11b, 11c, 15c, 18c, 19c, 20c, 21c and 27c). All of these unsolved instances have at least 24 orders and, except for instance 11b, six or eight batches. Based on these results we selected LBPT to rearrange the input data.

To evaluate the impact of the various lower bounds we compared nine versions of the branch-and-bound algorithm. We denote these versions by $V_1, V_2, \dots, V_9$. The versions refer to the lower bounds used. Each version can be combined with any initial upper bound or rearrangement strategy. As mentioned before, we use n runs of 2-opt to obtain the initial upper bound, and we use LBPT as the rearrangement strategy. The usage of lower bounds per version is given in Table 4.

If lower bound LB_k is used, then all the lower numbered lower bounds $LB_1, \dots, LB_{k-1}$ are also used. For LB_1 either LB_{1e} or a combination of LB_{1c} and LB_{1e} is used. In the latter case, which is denoted by $LB_{1c} + LB_{1e}$, LB_{1e} is computed if condition (1) of Section 4.2.1 is satisfied. Only in the case of V_9 is LB_{1e} always computed when

Table 3. The impact of the rearrangement strategies

Test set	Values		CPU times (in seconds)			
	2-opt	Opt	RAND	LOPT	LBPT	2-opt
FR01a	341.56	341.56	2.32	0.48	0.49	0.98
FR04a	342.43	342.43	342.43*	16.91	10.81	2.06
FR04b	315.98	312.00	314.69*	87.29	58.99	1.67
FR10a	411.35	410.67	617.55	5.32	3.82	4.51
FR10b	363.87	362.31	362.77*	83.53	46.61	2.93
FR10c	313.37	313.00*	313.37*	313.00*	313.03*	1.65
FR11a	444.33	443.00	444.33*	666.17	132.59	10.17
FR11b	386.46	386.46*	386.46*	386.46*	386.46*	7.14
FR11c	318.53	317.72*	318.53*	318.53*	317.72*	5.07
FR12c	238.90	232.46	232.00*	1163.53	660.12	2.32
FR13c	256.30	256.30	256.30*	142.57	43.11	3.17
FR14c	343.45	342.23	343.45*	135.19	118.47	3.85
FA18a	394.75	392.89	393.09*	87.81	54.85	5.22
FA21b	186.64	184.53	186.64*	8.96	6.10	2.41
PR24c	309.37	308.00	309.37*	316.38	43.87	0.88
PR27a	373.20	373.20	425.45	11.30	4.45	4.62
PR27b	331.68	330.99	331.68*	78.50	34.55	3.09
PR27c	286.14	282.00*	286.14*	282.00*	282.00*	2.05
Number solved			20	50	52	

* Indicates objective function value for truncated branch-and-bound.

Table 4. Usage of the lower bounds per version

V_1 : LB_{1e}
V_2 : $LB_{1c} + LB_{1e}$
V_3 : LB_2, LB_{1e}
V_4 : $LB_2, LB_{1c} + LB_{1e}$
V_5 : LB_3, LB_2, LB_{1e}
V_6 : $LB_3, LB_2, LB_{1c} + LB_{1e}$
V_7 : $LB_4, LB_3, LB_2, LB_{1e}$
V_8 : $LB_4, LB_3, LB_2, LB_{1c} + LB_{1e}$
V_9 : $LB_4, LB_3, LB_2, LB_{1e}, LB_{1c}$

$LB_{1c} < UB$. This may be worthwhile to improve the impact of LB_3 and LB_4 . Note that V_9 is the version that was used for testing the rearrangement strategies in Table 3.

Some typical test results are presented in Table 5. V_1 and V_2 , that only use LB_1 , perform poorly compared to the other versions. If only LB_1 or LB_2 is used (V_1, V_2, V_3 and V_4), then $LB_{1c} + LB_{1e}$ on average performs slightly better than LB_{1e} (V_2 and V_4 versus V_1 and V_3 respectively). However, using LB_3 and LB_4 it seems worthwhile to evaluate LB_{1e} more often, in particular for larger instances (V_5 and V_7 versus V_6 and V_8 respectively). V_9 combines the advantages of V_8 (the faster LB_{1c}) and V_7 (the tighter LB_{1e}). V_9 performs on average considerably better than the other versions. It performs slightly worse for only a few instances.

The impact of the lower bounds on the fathoming of the nodes is illustrated in Table 6. This table corresponds to the results for V_9 in Table 5. The column *nodes* gives the total number of nodes that has been evaluated (the

root not included). The columns LB_3 , LB_4 , LB_{2c} and LB_{2e} give the percentage of these nodes that are fathomed because of the associated lower bound. LB_{2e} denotes LB_2 using LB_{1e} , while LB_{2c} denotes LB_2 using first LB_{1c} and, if the node is not fathomed, also LB_{1e} . The last column gives the percentage of nodes that could not be fathomed. This includes the nodes at level n . A node is most frequently fathomed by LB_3 . The impact of LB_{2e} seems small, but from Table 5 we see that computing LB_{2e} speeds up the algorithm for most instances. This can be explained because it also tightens LB_3 and LB_4 .

The test results show that in general the additional computational effort per node to tighten a lower bound is compensated by the smaller number of nodes that must be evaluated. V_9 , in which all lower bounds are used, and where LB_{1e} is verified whenever $LB_{1c} < UB$, performs best.

7. Computational results

Based on the preliminary test results in Section 6, we have chosen version 9 of the branch-and-bound algorithm, using n runs of the 2-opt heuristic to obtain an initial upper bound, and then use LBPT to rearrange the input data. This implementation has been tested extensively by taking 50 instances for every parameter setting of Table 2. Because of the large number of instances (3150), we limited the maximum CPU time per instance to 600 seconds. The results are summarized in Tables 7 and 8. The tables contain a section per characteristic of the test sets.

Table 5. The impact of the lower bound

Test set	CPU times (in seconds) for version								
	V_1	V_2	V_3	V_4	V_5	V_6	V_7	V_8	V_9
FR01a	3.74	3.09	2.43	0.58	0.54	0.46	0.52	0.37	0.49
FR04a	342.43*	1419.10	28.19	23.09	13.99	12.49	11.90	10.66	10.81
FR04b	315.60*	314.03*	137.83	107.56	73.65	61.84	69.10	57.92	58.99
FR10a	411.35*	411.35*	378.95	390.09	169.86	175.33	3.68	29.61	3.82
FR10b	363.87*	363.87*	362.31*	362.31*	710.89	732.46	44.99	83.91	46.61
FR10c	313.37*	313.37*	313.00*	313.00*	313.03*	313.03*	313.03*	313.03*	313.03*
FR11a	444.33*	444.33*	244.99	190.47	183.07	151.42	154.44	129.59	132.59
FR11b	386.46*	386.46*	386.46*	386.46*	386.46*	386.46*	386.46*	386.46*	386.46*
FR11c	318.53*	318.53*	318.42*	317.72*	317.72*	317.72*	317.72*	317.72*	317.72*
FR12c	238.90*	238.90*	1577.85	1217.66	946.23	770.08	788.67	664.39	660.12
FR13c	256.30*	256.30*	208.26	168.79	89.13	92.21	48.92	56.28	43.11
FR14c	342.23*	342.23*	1474.46	1230.82	411.74	415.42	126.10	158.71	118.47
FA18a	394.75*	394.75*	266.61	247.86	174.06	168.16	54.35	59.15	54.85
FA21b	98.84	85.43	19.57	23.36	11.98	17.81	6.79	15.27	6.10
PR24c	309.37*	309.37*	943.71	760.14	222.24	215.00	44.84	53.96	43.87
PR27a	627.61	516.26	10.50	8.59	7.38	6.50	5.01	4.45	4.45
PR27b	330.99*	1681.65	75.81	58.28	52.54	42.74	41.30	33.90	34.55
PR27c	286.14*	283.32*	282.00*	282.00*	282.00*	282.00*	282.00*	282.00*	282.00*

* Indicates objective function value for truncated branch-and-bound.

Table 6. Number of nodes and the fathoming of nodes

Test set	Nodes	Nodes fathomed by				
		LB_3	LB_4	LB_{2c}	LB_{2e}	None
		(in %)				
FR01a	950	20	6	34	6	34
FR04a	95 908	45	5	18	6	26
FR04b	547 997	43	2	28	5	21
FR10a	40 350	25	34	0	7	35
FR10b	718 308	42	21	3	9	26
FR10c	26 476 064	38	14	14	10	24
FR11a	368 494	22	8	31	6	34
FR11b	5 427 591	20	4	46	5	26
FR11c	6 439 540	19	4	54	6	18
FR12c	6 309 341	34	5	38	3	21
FR13c	534 396	49	11	23	0	18
FR14c	1 358 899	59	9	13	2	17
FA18a	352 362	17	31	7	12	34
FA21b	68 400	28	18	19	10	26
PR24c	2 299 998	59	16	6	1	17
PR27a	30 742	23	16	24	5	34
PR27b	269 210	23	10	37	5	25
PR27c	16 092 284	24	5	49	4	18
Total (in %)	100	35	15	23	6	22

branch-and-bound: V_9 , using 2-opt and LBPT.

Except for the first section (the problem size), the results per section are arranged such that instances with the same number of batches can easily be compared.

The first column specifies the test set. Column “ $UB = OPT$ ” gives the percentage of instances for which the initial upper bound was already optimal. Column “ $max\ UB - OPT$ ” gives the maximum percentage difference between the initial upper bound and the optimal solution. Column “ OPT ” gives the percentage of instances for which the optimal solution was found within the maximum of 600 CPU seconds. The next three columns present CPU times for those instances that were solved to optimality. The average, minimum and maximum CPU times are given. Using a branch-and-bound algorithm, it is not necessarily time consuming to find the optimal solution, but it may be hard to prove optimality. Often a large portion of the CPU time is spent with no further improvement in the solution. To study this, the next two columns give the maximum and the average CPU time at which the best solution was found in the branch-and-bound algorithm. The next five columns are related to the fathoming of nodes, as in Table 6. The last row of Table 8 summarizes some statistics over all instances. We discuss the results by characteristics of the test sets.

The problem size: for the smaller instances (e.g., 1ab, 22 and 25), 2-opt often gives the optimal solution. It also performs well for the other instances with a maximum difference from the optimal solution of less than 6%, and

on average only 2.2%. If we compare the various values for n and m (sets 1 to 9) it appears that the problems become harder with increasing n and in particular with increasing m . This is not unexpected, since if the number of batches m increases, the number of feasible solutions increases rapidly. Problems with $n \geq 21$ or $m \geq 5$ could not always be solved, while for sets 6d and 8c none of the instances were solved.

The warehouse layout: instances with a relatively small number of aisles (test set 10) turn out to be relatively easy. This can be explained by the fact that the impact of the composition of the batch on the travel time increases with an increasing number of aisles. With a small number of aisles the extraction times will be the significant factor. Moreover, the complexity of the algorithm of Ratliff and Rosenthal (1983) depends on the number of aisles.

The pick density: the impact of the number of picks is rather small. Instances with more picks seem to be more difficult on average. This may be due to the fact that the computation time for the algorithm of Ratliff and Rosenthal (1983) increases with the number of picks.

Family grouping: if family grouping is applied, an aisle will more often be traversed completely, which makes the batching problem easier with respect to the travel times. The difference between sets 6 and 12 is very illustrative. However, if the picks are more concentrated in a few aisles as in set 13, the instances may become harder again because the routing will be more dependent on the composition of a batch.

The storage policy: in the comparison of randomized storage and ABC-storage (sets 6, 12, 14 and 15 versus sets 18, 19, 20 and 21) the latter appear to be more difficult. Again the most probable explanation is that for randomized storage an aisle is more often traversed completely.

The contribution of the extraction times: it seems that instances get more difficult with increasing extraction times. This may be explained as follows. If the extraction times are relatively small, the lead time of a batch is mainly determined by the travel time. The travel time may in many cases be determined by only a few orders in a batch. Thus, with smaller extraction times, a good indication of the batch lead time may be obtained while only a few orders are assigned to a batch. This also increases the impact of LB_2 .

Practical settings: these instances are clearly relatively easy. Considering the results for the other parameter settings this can be expected, since the practical instances are based on several favorable settings, such as a small number of picks and a small number of aisles.

General aspects: another interesting statistic is the time at which the best solution is found. As we can see, in particular the average time at which the best solution is found is very small. For all settings it is less than 12 CPU seconds, while overall it is even less than 4 CPU seconds. This is a phenomenon which is typical for many

Table 7. The results for the extensive test set (1)

Test set	UB = OPT	Max UB-OPT	OPT	CPU optimal			CPU best		Nodes fathomed by				
				Avg	Min	Max	Max	Avg	LB ₃	LB ₄	LB _{2c}	LB _{2e}	None
(in %)				(in CPU seconds)					(in %)				
The problem size													
FR01a	84	1.84	100	0.36	0.07	0.88	0.34	0.01	28	6	25	6	35
FR01b	92	1.76	100	15.63	0.59	334.52	5.93	0.12	18	1	33	12	37
FR02	76	1.50	100	5.79	0.33	117.09	1.22	0.02	23	3	32	8	34
FR03a	68	0.97	100	1.13	0.16	3.28	1.16	0.02	29	15	17	5	34
FR03b	58	2.56	82	144.88	9.35	534.24	46.00	0.92	24	2	43	7	24
FR04a	32	1.61	100	22.42	2.47	68.58	17.68	0.35	33	12	24	6	26
FR04b	26	2.77	98	149.33	4.82	523.33	157.09	3.14	34	6	33	6	22
FR05	8	2.07	10	334.76	144.95	477.54	200.35	4.01	29	2	44	6	19
FR06a	28	1.37	100	14.22	0.81	98.30	23.03	0.46	26	32	7	3	33
FR06b	10	2.26	96	158.91	9.69	558.19	267.39	5.35	33	23	15	4	25
FR06c	0	2.01	6	388.97	247.78	488.67	576.42	11.53	39	6	32	5	18
FR06d	0	2.03	0	—	—	—	588.34	11.77	38	2	39	5	16
FR07	0	1.83	32	322.88	26.23	556.53	596.71	11.93	36	15	24	5	20
FR08a	2	1.61	100	84.36	3.95	330.38	262.04	5.24	22	40	3	1	33
FR08b	0	1.94	2	455.35	455.35	455.35	589.90	11.80	33	30	14	3	20
FR08c	0	1.48	0	—	—	—	385.02	7.70	38	20	21	4	17
FR09	0	2.06	12	257.68	130.18	335.15	599.22	11.98	25	43	5	2	25
The warehouse layout													
FR06a	28	1.37	100	14.22	0.81	98.30	23.03	0.46	26	32	7	3	33
FR10a	18	1.06	100	2.11	0.15	10.48	6.05	0.12	24	30	2	10	34
FR11a	40	1.27	96	202.00	12.29	535.99	559.39	11.19	24	21	16	5	33
FR06b	10	2.26	96	158.91	9.69	558.19	267.39	5.35	33	23	15	4	25
FR10b	12	1.53	100	7.42	0.63	43.31	11.26	0.23	31	27	8	8	26
FR11b	0	1.43	6	439.73	316.08	581.89	584.54	11.69	29	13	27	5	25
FR06c	0	2.01	6	388.97	247.78	488.67	576.42	11.53	39	6	32	5	18
FR10c	12	2.10	84	152.22	3.95	518.66	108.38	2.17	40	17	19	4	19
FR11c	0	0.00	0	—	—	—	0.00	0.00	33	4	41	5	18
The pick density													
FR12a	34	2.84	100	1.41	0.07	10.88	8.69	0.17	25	22	17	2	35
FR14a	32	2.85	100	2.03	0.41	7.90	3.89	0.08	27	27	9	2	34
FR15a	54	2.88	100	1.00	0.05	4.69	2.25	0.05	20	16	26	4	34
FR12b	20	3.22	98	13.06	0.32	151.82	15.57	0.31	31	12	25	2	31
FR14b	28	3.62	96	12.88	0.33	87.07	10.24	0.20	32	13	18	8	30
FR15b	42	2.76	98	14.02	0.39	346.74	14.69	0.29	25	6	30	6	34
FR12c	18	4.03	80	122.81	1.77	491.30	179.44	3.59	37	8	30	2	24
FR14c	12	5.65	64	202.58	4.14	570.24	155.02	3.10	46	10	20	2	22
FR15c	26	4.62	72	107.98	0.02	516.47	343.83	6.88	25	6	39	5	25
Family grouping													
FR06a	28	1.37	100	14.22	0.81	98.30	23.03	0.46	26	32	7	3	33
FR12a	34	2.84	100	1.41	0.07	10.88	8.69	0.17	25	22	17	2	35
FR13a	38	3.27	100	8.87	0.29	248.28	63.27	1.27	23	10	22	5	40
FR06b	10	2.26	96	158.91	9.69	558.19	267.39	5.35	33	23	15	4	25
FR12b	20	3.22	98	13.06	0.32	151.82	15.57	0.31	31	12	25	2	31
FR13b	34	2.93	96	10.47	0.44	76.96	37.13	0.74	25	8	30	2	36
FR06c	0	2.01	6	388.97	247.78	488.67	576.42	11.53	39	6	32	5	18
FR12c	18	4.03	80	122.81	1.77	491.30	179.44	3.59	37	8	30	2	24
FR13c	14	2.76	52	143.55	15.84	494.74	131.66	2.63	33	13	26	2	25

Table 8. The results for the extensive test set (2)

Test set	UB = OPT	Max UB-OPT	OPT	CPU optimal			CPU best		Nodes fathomed by				
				Avg	Min	Max	Max	Avg	LB ₃	LB ₄	LB _{2c}	LB _{2e}	None
(in %)				(in CPU seconds)					(in %)				
The storage policy: randomized storage versus ABC storage													
FR06a	28	1.37	100	14.22	0.81	98.30	23.03	0.46	26	32	7	3	33
FA18a	42	1.35	100	29.93	3.24	89.02	43.67	0.87	25	25	7	9	33
FR12a	34	2.84	100	1.41	0.07	10.88	8.69	0.17	25	22	17	2	35
FA19a	42	2.16	100	2.23	0.13	18.91	14.88	0.30	23	25	13	4	34
FR14a	32	2.85	100	2.03	0.41	7.90	3.89	0.08	27	27	9	2	34
FA20a	32	2.71	100	2.05	0.22	9.61	5.09	0.10	28	30	8	1	34
FR15a	54	2.88	100	1.00	0.05	4.69	2.25	0.05	20	16	26	4	34
FA21a	52	3.22	100	1.55	0.11	11.08	5.97	0.12	19	15	19	12	34
FR06b	10	2.26	96	158.91	9.69	558.19	267.39	5.35	33	23	15	4	25
FA18b	8	1.57	72	257.05	25.70	590.47	476.82	9.54	31	19	13	12	25
FR12b	20	3.22	98	13.06	0.32	151.82	15.57	0.31	31	12	25	2	31
FA19b	40	2.24	98	10.97	0.53	48.00	21.57	0.43	25	13	21	6	35
FR14b	28	3.62	96	12.88	0.33	87.07	10.24	0.20	32	13	18	8	30
FA20b	16	2.63	100	20.59	1.11	190.10	153.76	3.08	39	22	12	2	26
FR15b	42	2.76	98	14.02	0.39	346.74	14.69	0.29	25	6	30	6	34
FA21b	48	2.36	98	9.46	0.47	95.92	31.31	0.63	26	7	19	19	30
FR06c	0	2.01	6	388.97	247.78	488.67	576.42	11.53	39	6	32	5	18
FA18c	0	0.91	0	—	—	—	531.88	10.64	37	6	24	14	18
FR12c	18	4.03	80	122.81	1.77	491.30	179.44	3.59	37	8	30	2	24
FA19c	12	2.94	62	131.44	13.83	503.28	46.78	0.94	36	10	25	6	22
FR14c	12	5.65	64	202.58	4.14	570.24	155.02	3.10	46	10	20	2	22
FA20c	14	3.10	70	155.84	0.07	595.64	468.03	9.36	50	9	20	2	19
FR15c	26	4.62	72	107.98	0.02	516.47	343.83	6.88	25	6	39	5	25
FA21c	20	2.36	56	125.68	0.02	542.94	266.36	5.33	27	9	21	17	27
The contribution of the extraction times													
FR06a	28	1.37	100	14.22	0.81	98.30	23.03	0.46	26	32	7	3	33
FR16a	38	1.65	100	3.50	0.23	20.11	9.54	0.19	16	15	27	7	34
FR17a	4	1.28	100	63.40	4.96	473.62	289.83	5.80	31	32	2	1	33
FR06b	10	2.26	96	158.91	9.69	558.19	267.39	5.35	33	23	15	4	25
FR16b	22	2.15	100	28.91	1.03	195.86	125.75	2.52	18	7	42	7	26
FR17b	0	1.38	44	270.67	67.41	562.51	593.39	11.87	41	29	4	1	25
FR06c	0	2.01	6	388.97	247.78	488.67	576.42	11.53	39	6	32	5	18
FR16c	4	2.87	40	378.59	140.64	588.40	576.94	11.54	18	3	53	7	19
FR17c	0	1.03	0	—	—	—	527.69	10.55	60	8	12	2	18
Practical settings													
PR22	70	2.62	100	0.09	0.03	0.56	0.09	0.00	44	13	6	3	34
PR23	38	1.77	100	1.96	0.03	18.23	4.13	0.08	46	21	6	2	26
PR24a	2	2.39	100	11.98	0.33	449.67	436.59	8.73	49	17	0	0	33
PR24b	8	1.78	100	19.09	0.64	338.13	68.20	1.36	43	29	2	1	25
PR24c	12	3.30	90	141.59	8.05	487.80	66.94	1.34	53	17	8	4	18
PR25	78	1.33	100	0.28	0.06	1.33	0.18	0.00	19	3	33	10	36
PR26	50	3.14	100	11.96	0.82	80.46	11.18	0.22	25	5	34	8	28
PR27a	32	1.77	100	4.43	0.24	15.14	7.64	0.15	21	21	19	5	34
PR27b	24	2.47	98	40.44	0.89	171.71	47.44	0.95	25	12	29	7	26
PR27c	4	2.74	34	324.26	77.88	564.76	427.89	8.56	28	4	41	8	20
Total	24	2.20	75					3.73	38	14	22	5	21

branch-and-bound algorithms. Frequently it does not take much time to find the optimal solution. Most of the CPU time is spent to prove optimality. Therefore, for practical purposes a truncated branch-and-bound would do very well.

Finally we take a closer look at the lower bounds. The overall statistics are very similar to those of the smaller test set in Table 6. The individual settings show that the impact of the various lower bounds differs a lot. LB_2 is highly dependent on the travel time for a batch. LB_3 and LB_4 tighten LB_2 especially by taking extraction times better into account. Indeed relatively many nodes are fathomed because of LB_3 and LB_4 if the extraction times play a more important role. This can be seen if we compare, e.g., sets 17 and 16, where the extraction times in set 17 are much larger, and sets 14 and 15, where the number of picks and thus the total extraction time is much larger in set 14.

8. Conclusions

We have presented a branch-and-bound algorithm for the order batching problem for wave picking in a parallel-aisle warehouse. Starting from a basic version of the algorithm, we considered several improvements. In particular, we have studied the impact of the initial upper bound, several lower bounds, and a preprocessing step, on the performance of the algorithm. Using n (the number of orders) runs of a 2-opt heuristic gave excellent upper bounds. A major improvement on the performance of the branch-and-bound algorithm was achieved by a preprocessing step, which consists of rearranging the input data. Another significant improvement was achieved by tightening the lower bound. It turned out that additional CPU time per node to compute tighter lower bounds was clearly compensated by a reduction of the number of nodes that need to be evaluated.

After tuning the algorithm, it has been tested on a large set of problem instances using various parameter settings. From the test results it followed that several parameters have a significant impact on the running time for the algorithm. In particular instances become harder to solve with an increasing number of batches and orders. In addition the warehouse layout, the number of picks and the distribution of picks are also important. Instances appear to be easier when it is more likely that an aisle will be traversed completely, e.g., if we compare family grouping with no family grouping or randomized storage with ABC-storage. If extraction times contribute more to batch lead times, instances become harder. The practical instances turned out to be relatively easy.

Most instances with up to 24 orders and four batches can be solved within a minute, and on average these

instances are solved even much faster. Since the 2-opt heuristic gives excellent upper bounds, the combination of 2-opt with a truncated branch-and-bound will give a very robust and excellently performing heuristic often with optimal solutions. Since on average no improvement was found after more than 12 seconds, a limited running time for the branch-and-bound algorithm of 1 minute will do very well. With the rapidly increasing computational speed of current Personal Computers this gives a fine tool for solving order batching problems in practice, now and in the future. However, since larger instances are still hard to solve, further research on improvements of the lower bounds and perhaps of the rearrangement methods is necessary.

References

- Armstrong, R.D., Cook, W.D. and Saipet, A.L. (1979) Optimal batching in a semi-automated order picking system. *Journal of the Operational Research Society*, **30**(8), 711–720.
- Choe, K.-I. (1991) Aisle-based order pick systems with batching, zoning and sorting. PhD thesis, Georgia Institute of Technology, Atlanta, GA.
- de Koster, R., van der Poort, E. and Wolters, M. (1999) Efficient order batching methods in warehouses. *International Journal of Production Research*, **37**(7), 1479–1504.
- Elsayed, E.A. (1981) Algorithms for optimal material handling in automatic warehousing systems. *International Journal of Production Research*, **19**(5), 525–535.
- Elsayed, E.A. and Lee, M.-K. (1996) Order processing in automated storage/retrieval systems with due dates. *IIE Transactions*, **28**(7), 567–577.
- Elsayed, E.A., Lee, M.-K., Kim, S. and Scherer, E. (1993) Sequencing and batching procedures for minimizing earliness and tardiness penalty of order retrievals. *International Journal of Production Research*, **31**(3), 727–738.
- Elsayed, E.A. and Stern, R.G. (1983) Computerized algorithms for order processing in automated warehousing systems. *International Journal of Production Research*, **21**(4), 579–586.
- Elsayed, E.A. and Unal, O.I. (1989) Order batching algorithms and travel-time estimation for automated storage/retrieval systems. *International Journal of Production Research*, **27**(7), 1097–1114.
- Garey, M.R. and Johnson, D.S. (1979) *Computers and Intractability: A Guide to the Theory of NP-completeness*, W.H. Freeman and Company, New York.
- Gibson, D.R. and Sharp, G.P. (1992) Order batching procedures. *European Journal of Operational Research*, **58**(1), 57–67.
- Hwang, H., Baek, W. and Lee, M.-K. (1988) Clustering algorithms for order picking in an automated storage and retrieval system. *International Journal of Production Research*, **26**(2), 189–201.
- Hwang, H. and Lee, M.-K. (1988) Order batching algorithms for a man-on-board automated storage and retrieval system. *Engineering Costs and Production Economics*, **13**, 285–294.
- Pan, C.-H. and Liu, S.-Y. (1995) A comparative study of order batching algorithms. *Omega*, **23**(6), 691–700.
- Ratliff, H.D. and Rosenthal, A.S. (1983) Order-picking in a rectangular warehouse: a solvable case of the traveling salesman problem. *Operations Research*, **31**(3), 507–521.
- Rosenwein, M.B. (1996) A comparison of heuristics for the problem of batching orders for warehouse selection. *International Journal of Production Research*, **34**(3) 657–664.

Biographies

Noud Gademann graduated in 1987 and received his Ph.D. in 1993, both in Applied Mathematics at the University of Twente in The Netherlands. The title of his Ph.D. thesis was Linear Optimization in Random Polynomial Time. From 1993 to 1998 he worked as an Assistant Professor in Production and Operations Management at the University of Twente. Currently he works as a Research Assistant at the Center for Production, Logistics and Operations Management of the University of Twente and as a consultant at ORTEC Consultants in Gouda, The Netherlands. His interests include combinatorial optimization, warehouse management, and production and project management.

Jeroen P. van den Berg received his M.Sc. in Applied Mathematics and Ph.D. in Mechanical Engineering at the University of Twente in The Netherlands. The title of his Ph.D. thesis was Planning and Control of Warehousing Systems. Since 1997 he has been a management consultant for Berenschot in Utrecht, The Netherlands. His interests include logistics, warehousing, supply chain management, IT and warehouse management systems.

Hassan H. van der Hoff received his M.Sc. in Applied Mathematics in 1995. The title of his masters thesis was Order Batching in a Warehouse. He is currently a consultant at Holland Railconsult.

Contributed by the Scheduling Department